

Probing Optimizer Dynamics in Grokking Setting

Jean Ciardo, Joshua Oyebanji, Mathieu Verest
Department of Computer Science, EPFL, Switzerland

Abstract—We study how optimizer geometry affects grokking in modular addition. Keeping the task, architecture, and data split fixed, we compare AdamW with two matrix-normalized optimizers, Muon and EGD. We find that matrix-normalized updates can make grokking faster and occur over a wider hyperparameter range. Fourier analysis shows that fast no-momentum runs generalize through wider circuits than AdamW, suggesting that optimizer choice affects both grokking speed and the learned mechanism.

I. INTRODUCTION

The phenomenon of Grokking[1] is a delayed generalization in which a model first quickly memorizes the training set and only later learns a solution that generalizes. In modular addition, this transition is associated with the emergence of a Fourier-based circuit, making the task a useful setting for studying learning dynamics [2]. Since its discovery, this phenomenon has been extensively studied using very different approaches, but we found that even modern review, lack systematic study of different optimizers and their effect on the training and generalization dynamics [3], [4]. In this work, we study how the choice of optimizer may affects grokking. We compare AdamW[5] with two matrix-normalized optimizers, Muon[6] and Egalitarian Gradient Descent (EGD)[7], while keeping the task, architecture, and data split fixed. We evaluate how learning rate, weight decay, momentum, and the orthogonalization method affect grokking speed and circuit formation. Our results show that matrix-normalized optimizers can expand the grokking hyperparameter region, accelerate generalization, and produce wider Fourier circuits. This suggests that optimizer geometry shapes not only how fast grokking occurs, but also the mechanism by which the generalizing solution emerges.

II. EXPERIMENTAL SETUP

A. Task and baseline

We take the modular-addition grokking setup of Nanda et al. [2] as our reference point, a configuration in which grokking is known to occur reliably and has been mechanistically characterised. Throughout this work we keep the *task* and *architecture* fixed. This stable baseline is what we hold fixed as we substitute alternative optimizers in the rest of the paper. This controlled setup reduces the confounds attributed at the task and architecture level, allowing us to compare how different optimizers and their associated hyperparameters shape the learning dynamics.

a) *Task and data.*: The model performs modular addition $a + b \bmod P$ with $P = 113$. Each input is a sequence “ $a\ b =$ ”, where $a, b \in \{0, \dots, P - 1\}$ are one-hot tokens and “ $=$ ” is a read-out token at whose position the model predicts $c =$

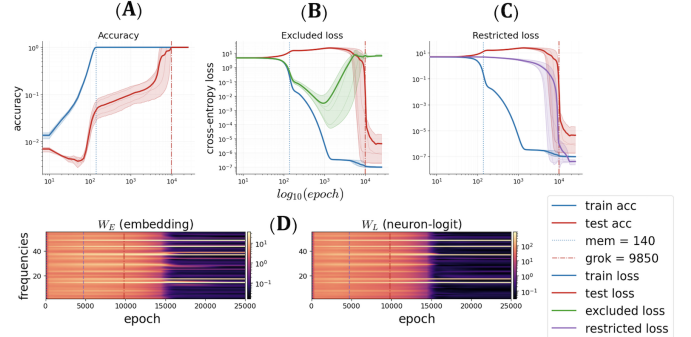


Fig. 1. **AdamW baseline (Nanda et al. [2]):** 5 seeds, 25,000 epochs. *mem/grok*: first epoch where mean train/test accuracy crosses 0.99. (A) Train/test accuracy (log-log). (B) Excluded loss: train Cross-entropy after removing the key-frequency logit projection. (C) Restricted loss: Cross-entropy on the full data using only that key-frequency projection; (D) Per-frequency masses ($k = 1 \dots 56$) for a single seed at W_E, W_L

$a + b \bmod P$; this is a classification over $P^2 = 12,769$ input pairs. Following [2], we train in full batch on a fixed random 30% of the pairs and evaluate on the remaining 70%.

b) *Architecture and reference regime.*: We use the same unmodified one-layer ReLU transformer as [2]: embedding dimension $d = 128$ with learned positional embeddings, 4 attention heads of dimension $d/4 = 32$, and an MLP with $n = 512$ hidden units, with no LayerNorm. Our reference runs use AdamW [5] c.f. Fig.1. This regime reproduces the canonical grokking signature: train accuracy reaches ≈ 1 rapidly while test accuracy stays at chance, followed by a delayed generalisation transition (mean $T_{\text{grok}} \approx 10,000$).

B. Fourier analysis and metrics

We want to compare how each optimizer reaches the generalising solution. We therefore reuse the mechanistic toolkit of Nanda et al. [2] and turn the components of the reverse-engineered circuit into progress measures we track during training.

a) *The circuit*: The one-layer transformer solves $a + b \bmod P$ by embedding inputs on a circle and adding angles: W_E maps tokens to $\sin/\cos(w_k a)$ at a sparse set of *key frequencies* $w_k = 2\pi k/P$, the layers combine them into $\sin/\cos(w_k(a+b))$ via trigonometric identities, and the unembedding reads off the logit for c as a sum of $\cos(w_k(a+b-c))$ that peaks at $c \equiv a + b \bmod P$ [2]. The mechanism relies on a subset of frequencies, making the Fourier domain an intuitive space tool to measure circuit formation.

b) *Spectral concentration*: We take the Fourier transform of the neuron logit map $W_L = W_U W_{\text{out}}$ along the input dimension and compute the squared ℓ_2 mass of each positive

frequency $k = 1, \dots, (P-1)/2$, using the symmetry between k and $P-k$ to discard the redundant half of the spectrum.

Two measures summarise the resulting distribution. The *key-mass concentration* is the fraction of mass on the key frequencies; it rises toward 1 as the circuit forms, and we set the circuit marker T_{circuit} at its first 0.5 crossing. The *Gini coefficient* quantifies how concentrated the mass is across frequencies (0: uniform, 1: maximally concentrated). We also record per-frequency *emergence* times to track when individual circuit frequencies appear over training.

c) *Excluded and restricted loss*: Still following [2], we isolate the circuit with two ablation losses as progress measures. *Excluded loss* is the train cross-entropy after removing the key frequencies’ projections: its through and recovery tracks the formation and later cleanup of the memorizing solution. *Restricted loss* is the cross-entropy on the full data using *only* the key frequencies’ projections, falling to zero once the circuit independently solves the task. Together they separate the memorizing and generalizing components of the network.

C. Optimizers

As stated we use AdamW as our reference optimizer, which is a natural baseline because it is the standard optimizer in many grokking experiments.

We then compare AdamW to a family of matrix-normalized optimizers: Muon [6] and Egalitarian Gradient Descent (EGD) [7]. Unlike AdamW, which rescales each parameter coordinate using historical second-moment estimates, Muon and EGD act on whole weight matrices. Given a matrix update direction $A = USV^\top$, they replace it with an approximately orthogonalized direction $\text{Orth}(A) = UV^\top$. This equalizes the singular values of the update, so the optimizer changes the geometry of learning at the matrix level rather than adapting individual coordinates independently.

These kinds of optimizers are good candidates for grokking tasks because the gradient may contain strong, high-signal directions that are useful for quickly reducing training loss but mainly correspond to memorization. By setting the singular values to 1, lower-signal directions that may lead toward grokking could receive more weight [7] [8].

Our choice of these two algorithms was motivated, first, by EGD being specifically constructed to make grokking faster, already tested in this setting, and state-of-the-art on this grokking task in [7]. Muon was chosen because of its similarity to EGD and because it has recently gained a lot of traction due to its performance when training LLMs [9].

We are especially interested in the potential effect of applying momentum before orthogonalization, as in Muon, or after orthogonalization, as in EGD. We are also interested in the effect of the chosen orthogonalization method: SVD or randomized SVD (see Appendix A for pseudo-code) for EGD, and Newton-Schulz iteration [10] for Muon. Muon with no momentum is equivalent to EGD with no momentum, up to the method used to obtain $\text{Orth}(G)$, where G is the gradient matrix.

We find this interesting because Newton-Schulz iteration is cheaper than SVD and less restrictive than randomized SVD, which requires the gradient matrix to be low-rank in order to produce a good approximation of $UV^\top$. Therefore, Muon without momentum has the potential to be a good fast-grokking optimizer at a lower computational cost than EGD.

III. RESULTS

A. Hyperparameter sweep

For every optimizer we vary the learning rate, weight decay and momentum for EGD and muon over a logarithmic grid, with two seeds per configuration (Fig. 3). We make several direct conclusions:

Grokking occurs over a wide hyperparameter range, but the range depends on the optimizer. Given they reach the full test accuracy within the budget, the fraction of configurations that eventually grok is roughly twice as large for matrix-normalized optimizers without momentum (36% for Muon without momentum and EGD, as for AdamW (16%); Muon with momentum sits in between (28%). In this sense gradient orthogonalization *expands* the grokking region in hyperparameter space. Consistent with our remark below on momentum, Muon with momentum is also the only family whose grokking-time landscape is not monotone around its optimum.

Weight decay controls the speed of the transition. For EGD and Muon without momentum, at the best learning rate, the median grokking epoch decreases monotonically with weight decay until $wd = 0.1$ (e.g. EGD $m=0$ at $lr=0.3$: a $\times 10$ increase in wd translates roughly to a $\times 4$ acceleration, until the magnitude of regularization prevents memorization altogether ($wd = 1$). This matches the known role of regularization in delayed generalization [1],[2].

Orthogonalization can partially compensate for removing hidden-matrix weight decay. With $wd = 0$ on the hidden matrices, AdamW never groks within 25k epochs, whereas the matrix-normalized optimizers still do, even if slowly ($T_{\text{grok}} \sim 3.5\text{k} - 11\text{k}$; 8 of 49 $wd=0$ runs). We interpret this as a kind of implicit regularization effect of spectrally normalized updates. *Caveat*: in our hybrid setup the embeddings and biases are always trained by AdamW with $wd = 1$, so these runs are not fully free of decay; the statement concerns the hidden 2D parameters only.

B. Choice of dynamical Regimes

For the dynamics study we select two regimes per optimizer: the *fast* regime, configuration that groks earliest in the sweep (e.g. ~ 200 epochs for EGD $m=0$ and Muon $m=0$), and a *slow* regime closer to the baseline of (Nanda et al. 2023)[2], grokking around a few thousand epochs. Slow configurations are obtained from the fast ones by lowering *only* the learning rate: among the swept hyperparameters it is the one least likely to introduce a geometric bias, in contrast to weight decay, which directly shapes the norm dynamics that grokking depends on. Each configuration is run with 5 seeds; all timing markers below are computed per seed and aggregated afterwards.

C. Grokking signatures are present even in the fastest regimes

Across seeds, every configuration shows coherent and clearly distinct dynamics. As noted in the literature, some hyperparameter settings generalize directly without a grokking phase; one could expect our fastest configurations ($T_{\text{grok}} \approx 170$ epochs, vs. $\sim 7,750$ for the Nanda baseline) to be of this type. It is not the case: even the fastest runs display the characteristic signatures; a memorization phase ($T_{\text{mem}} \approx 30\text{--}60$ epochs, train accuracy 0.99 while test accuracy is at chance), a delayed test accuracy transition, an excluded loss dip, and a restricted loss that, in the end, represent the full solution. The models in all configurations become *dependent* on a set of key frequencies to solve the task (Fig. 4).

The depth of the excluded-loss dip, however, varies greatly and systematically Fig. ??). In every slow configuration and in all momentum configurations, the excluded loss reaches ≈ 0 : at its peak memorization moment, meaning the model can fit the training set *entirely* through frequencies outside the key-frequency subspace. By contrast, in the fast matrix normalized runs without momentum, it never does (minimum excluded loss 0.74 ± 0.13 for Muon $m=0$ fast, 0.16 for EGD $m=0$ fast, vs. $0.14\text{--}0.53$ for AdamW fast and ≤ 0.01 elsewhere): the memorization setting never reaches full intensity. We propose two compounding explanations: (i) A race argument; let T_{mem} and T_{gen} denote the epochs at which train and test accuracy first exceed 99%. In these two runs generalization follows memorization almost immediately — $T_{\text{gen}} \approx 3 \cdot T_{\text{mem}}$ (3.25 ± 0.28 for Muon $m=0$ fast, 2.97 ± 0.24 for EGD $m=0$ fast), the smallest ratio of any configuration in the grid search. By contrast, every run whose excluded loss reaches ≈ 0 takes much longer to generalize relative to memorizing (ratio ≥ 8 for the fast runs, $20\text{--}50$ for the slow runs). The memorization dynamic is therefore short-circuited by an early forming circuit rather than given the much larger horizon (thousands of epochs) the slow runs spend developing it. (ii) A *capacity* argument; these models recruit 11 to 17 of the 56 available frequencies into the circuit (next subsection), leaving fewer orthogonal frequency directions available for a memorizing solution. We recognize that part of this effect is by definition: using more key frequencies leaves fewer excluded frequencies and therefore mechanically raises the excluded loss. But this does not account for the whole phenomenon, because the excluded loss dip is also genuinely shallower, which suggests that memorization in the remaining directions is interrupted rather than simply measured on a smaller subspace.

D. The width of the circuit at generalization time depends on the optimizer geometry

Where the AdamW baseline (from [2]) forms its Fourier circuit on 5 essential frequencies, our two fastest configurations rely on far more: 17.4 ± 0.8 for Muon $m=0$ and 11.4 ± 0.5 for EGD $m=0$ (cumulative-energy criterion $\tau = 0.9$; details per seed in Appendix B). To compare circuit width across configurations *without* committing to any key frequency selection rule, we use the participation ratio of the normalized W_L frequency masses, $\text{PR} = 1/\sum_k p_k^2$, i.e. the effective

number of active frequencies ($\text{PR} = 56$ when uniform, $= 5$ for a canonical 5 frequency circuit). Figure 2 shows PR along the grok-normalized timeline t/T_{grok} .

From this figure we can visualize several things. First, *at memorization* every configuration is nearly indistinguishable: $\text{PR} \approx 46\text{--}55$; the spectra are quite homogeneous (the optimizers do not differ in the way they memorize). Then, *at the moment of grokking* ($t/T_{\text{grok}} = 1$) the configurations form three groups:

$$\underbrace{\text{PR} \approx 5\text{--}9}_{\text{AdamW (both), all slow}} < \underbrace{\text{PR} \approx 21}_{\text{EGD } m=0.9, \text{ Muon } m=0.95 \text{ fast}} < \underbrace{\text{PR} \approx 36}_{\text{EGD, Muon } m=0 \text{ fast}}$$

The canonical “sparse circuit at generalization” is therefore not a general property of grokking on this task: it is a property of AdamW and of the slow regime. Fast matrix normalized optimizers generalize while their neuron logit map is still spectrally *loosely distributed*, $\times 2$ as much when momentum is removed. AdamW acts as a control, from this we can deduce that it is not speed alone that produces wide circuits but the regime defined by the update orthogonalization with a high learning-rate.

The trajectories after grokking group in two distinct clusters. The momentum configurations (EGD $m=0.9$, Muon $m=0.95$ fast) compress their wide circuit down to the canonical $\text{PR} \approx 5$ during cleanup (therefore not a product of not of generalization itself). However the no-momentum fast configurations never compress: they plateau for the remaining 25k epochs, their L_2 norm basically stopping to decay after grokking. These runs reach a stable, high-norm, spectrally more distributed solution at $> 99\%$ test accuracy which seem to be a qualitative different endpoint.

The configuration Muon $m=0.95$ fast is interesting because it bridges the two dynamics. It carries both ingredients: orthogonalized updates at high learning rate and momentum and additionally exhibits both effects in sequence: like the no-momentum fast runs it stays spectrally distributed up to T_{grok} , then, like the other momentum configurations, it compresses to the canonical $\text{PR} \approx 5$ during cleanup. Notably the compression is simultaneous with grokking, way sharper than the gradual descent of EGD $m=0.9$. It thus isolates the two mechanisms as independent and additive: orthogonalization with a high learning rate *widens* the circuit at generalization, while momentum *drives its later compression*, the two being neither the same effect nor a byproduct of generalization itself.

IV. SUMMARY

We compared AdamW, Muon, and EGD in the modular-addition grokking setting. Matrix-normalized optimizers expanded the range of hyperparameters where grokking occurs and accelerated generalization. They also changed the learned Fourier circuits: fast no-momentum Muon and EGD runs generalized with wider frequency support than AdamW or slower regimes. Overall, our results suggest that optimizer geometry affects not only when grokking happens, but also how the generalizing solution forms.

REFERENCES

- [1] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra, “Grokking: Generalization beyond overfitting on small algorithmic datasets,” 2022. [Online]. Available: <https://arxiv.org/abs/2201.02177>
- [2] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt, “Progress measures for grokking via mechanistic interpretability,” 2023. [Online]. Available: <https://arxiv.org/abs/2301.05217>
- [3] T. Agrawal and M. Kumar, “Grokking in neural networks: A review,” *SN Computer Science*, vol. 6, p. 627, 2025. [Online]. Available: <https://doi.org/10.1007/s42979-025-04182-z>
- [4] S. B. Manir and A. P. Rupa, “A systematic empirical study of grokking: Depth, architecture, activation, and regularization,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.25009>
- [5] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” *arXiv preprint arXiv:1711.05101*, 2017.
- [6] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein, “Muon: An optimizer for hidden layers in neural networks,” 2024. [Online]. Available: <https://kellerjordan.github.io/posts/muon/>
- [7] A. S. Pasand and E. Dohmatob, “Egalitarian gradient descent: A simple approach to accelerated grokking,” 2026. [Online]. Available: <https://arxiv.org/abs/2510.04930>
- [8] J. Lee, B. G. Kang, K. Kim, and K. M. Lee, “Grokfast: Accelerated grokking by amplifying slow gradients,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.20233>
- [9] J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, Y. Chen, H. Zheng, Y. Liu, S. Liu, B. Yin, W. He, H. Zhu, Y. Wang, J. Wang, M. Dong, Z. Zhang, Y. Kang, H. Zhang, X. Xu, Y. Zhang, Y. Wu, X. Zhou, and Z. Yang, “Muon is scalable for llm training,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.16982>
- [10] Z. Kovarik, “Some iterative methods for improving orthonormality,” *SIAM Journal on Numerical Analysis*, vol. 7, no. 3, pp. 386–389, 1970. [Online]. Available: <https://doi.org/10.1137/0707031>

APPENDIX A ALGORITHM

Algorithm 1 Muon

Require: Learning rate η , momentum μ

- 1: Initialize $B_0 \leftarrow 0$
- 2: **for** $t = 1, \dots$ **do**
- 3: Compute gradient $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
- 4: $B_t \leftarrow \mu B_{t-1} + G_t$
- 5: $O_t \leftarrow \text{NewtonSchulz5}(B_t)$
- 6: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta O_t$
- 7: **end for**
- 8: **return** θ_t

Algorithm 2 Muon without momentum

Require: Learning rate η ,

- 1: Initialize $B_0 \leftarrow 0$
- 2: **for** $t = 1, \dots$ **do**
- 3: Compute gradient $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
- 4: $O_t \leftarrow \text{NewtonSchulz5}(G_t)$
- 5: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta O_t$
- 6: **end for**
- 7: **return** θ_t

Algorithm 3 EGD without momentum with optional RSVD

Require: Learning rate η , flag use_RSVD

- 1: **for** $t = 1, \dots$ **do**
- 2: Compute gradient $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
- 3: **if** use_RSVD **then**
- 4: $(U, S, V^{\top}) \leftarrow \text{RSVD}(G_t)$
- 5: **else**
- 6: $(U, S, V^{\top}) \leftarrow \text{SVD}(G_t)$
- 7: **end if**
- 8: $P_t \leftarrow US^{-1}U^{\top}$
- 9: $\tilde{G}_t \leftarrow P_t G_t$
- 10: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta \tilde{G}_t$
- 11: **end for**
- 12: **return** θ_t

Algorithm 4 EGD with momentum and optional RSVD

Require: Learning rate η , momentum coefficient β , flag

- use_RSVD
- 1: Initialize momentum buffer $B_0 \leftarrow 0$
- 2: **for** $t = 1, \dots$ **do**
- 3: Compute gradient $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
- 4: **if** use_RSVD **then**
- 5: $(U, S, V^{\top}) \leftarrow \text{RSVD}(G_t)$
- 6: **else**
- 7: $(U, S, V^{\top}) \leftarrow \text{SVD}(G_t)$
- 8: **end if**
- 9: $P_t \leftarrow US^{-1}U^{\top}$
- 10: $\tilde{G}_t \leftarrow P_t G_t$
- 11: $B_t \leftarrow \beta B_{t-1} + \tilde{G}_t$
- 12: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta B_t$
- 13: **end for**
- 14: **return** θ_t

APPENDIX B ADDITIONAL FREQUENCY-SELECTION RESULTS

Optimizer	Configuration	$\tau = 0.85$	$\tau = 0.90$	
AdamW	fast	3.2 ± 0.4 (3–4)	3.2 ± 0.4 (3–4)	3.
AdamW	slow	4.0 ± 0.6 (3–5)	4.6 ± 0.5 (4–5)	4.
Muon $m = 0$	fast	15.8 ± 0.7 (15–17)	17.4 ± 0.8 (17–19)	18.4
Muon $m = 0$	slow	4.6 ± 0.5 (4–5)	4.8 ± 0.4 (4–5)	5.
Muon $m = 0.95$	fast	4.6 ± 0.8 (4–6)	5.2 ± 1.2 (4–7)	5.
Muon $m = 0.95$	slow	4.8 ± 0.7 (4–6)	5.0 ± 0.9 (4–6)	5.
EGD $m = 0$	fast	10.8 ± 0.4 (10–11)	11.4 ± 0.5 (11–12)	12.6
EGD $m = 0$	slow	3.8 ± 0.7 (3–5)	4.6 ± 0.5 (4–5)	4.
EGD $m = 0.9$	fast	4.6 ± 0.5 (4–5)	5.0 ± 0.0 (5–5)	5.
EGD $m = 0.9$	slow	3.8 ± 0.7 (3–5)	4.0 ± 0.9 (3–5)	4.

TABLE I
FREQUENCY COUNTS UNDER CUMULATIVE-ENERGY THRESHOLDS.
VALUES ARE MEAN $\pm$ STANDARD DEVIATION OVER FIVE SEEDS;
PARENTHESES SHOW THE SEED RANGE.

APPENDIX C ADDITIONAL PLOTS

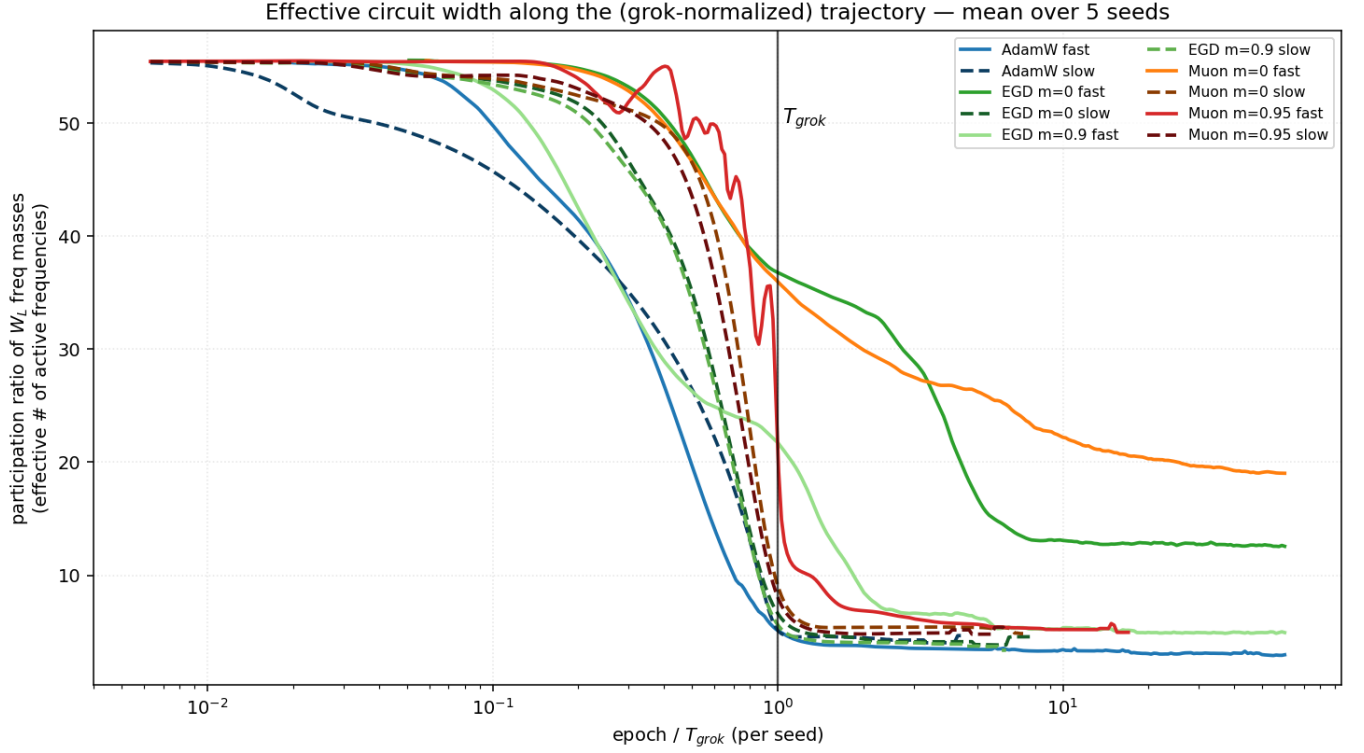


Fig. 2. **Participation ratio of W_L frequency masses vs epoch/ T_{grok} .** Per-seed normalization, mean over 5 seeds, $\log$ x-axis. Vertical line at T_{grok} . Solid = fast, dashed = slow regime.

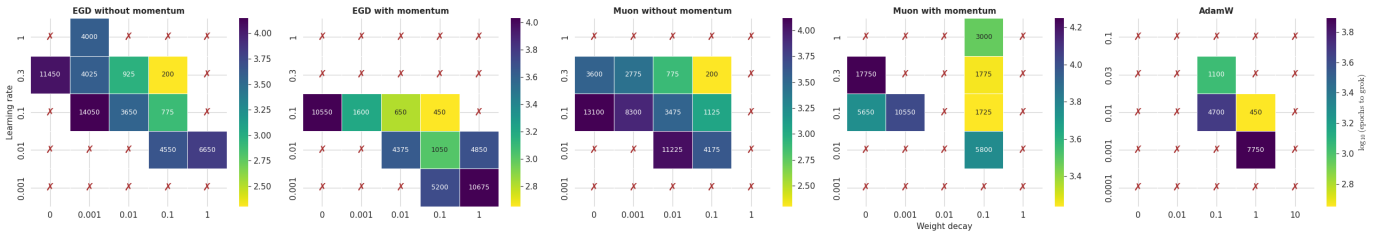


Fig. 3. Grokking speed across (learning rate, weight decay) configurations for each optimizer. Color encodes $\log_{10}$ of the average epoch to grok over 2 seeds. Momentum is set to its canonical value per optimizer: 0.9 for EGD (SGD-style), 0.95 for Muon (original Muon recipe [6]), and $(\beta_1, \beta_2) = (0.9, 0.98)$ for AdamW (following [2]). $\times$ = trained, did not grok; $\cdot$ = not tested.

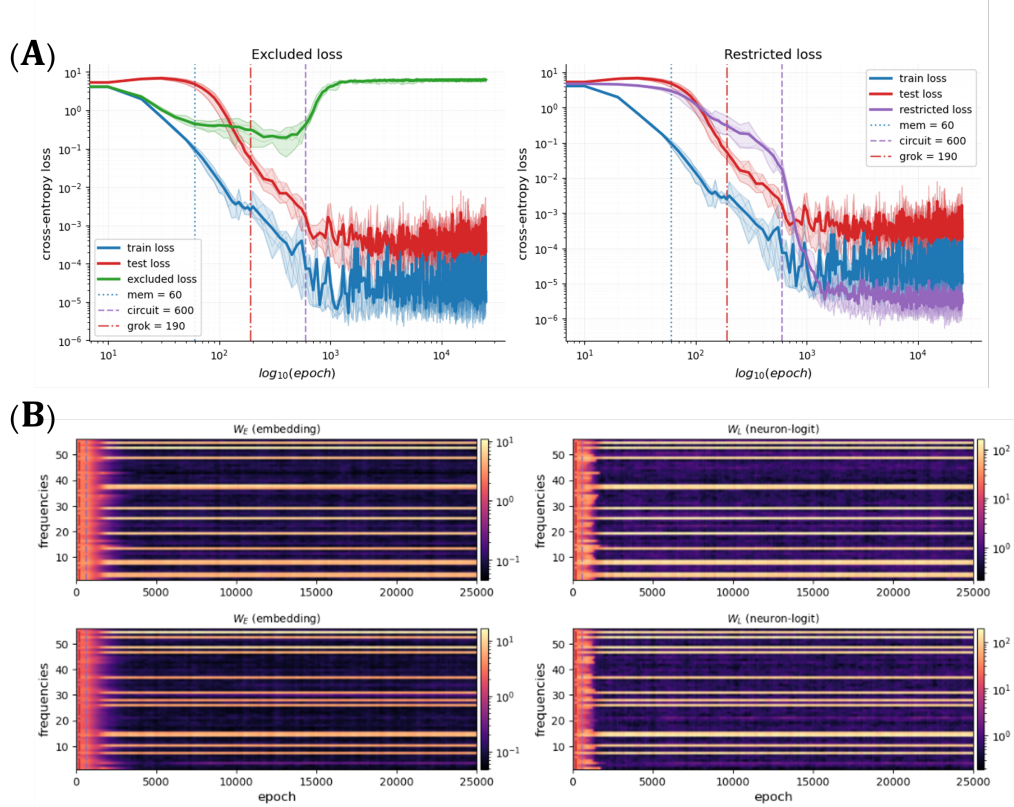


Fig. 4. **Losses and Fourier spectra for the fastest Muon configuration (no momentum, $\text{wd} = 0.1$, $\text{lr} = 0.3$);** (A) Train, test, excluded (left) and restricted (right) cross-entropy losses. The rise of the excluded loss and the collapse of the restricted loss onto the test loss show that the model becomes dependent on a small set of key frequencies. (B) Fourier spectrograms of W_E (left) and W_L (right) in the Nanda basis (56 frequencies), for two seeds (rows): a few characteristic frequencies clearly emerge and dominate throughout training.